

Fluid - a walkthrough

Fluid

A native macOS application, written entirely in Mojo, in which every kernel runs on the Apple GPU and there is no shader anywhere in the pipeline.

Drag the mouse and coloured dye swirls through a velocity field that is advected along itself and then made divergence-free by a pressure solve. It is Jos Stam's *Stable Fluids* (SIGGRAPH 1999), 320×240 of simulation magnified into a 960×720 window, running at 60 fps on an M4.

It is also the reason this fork can say anything trustworthy about GPU launch cost, which is the part of the story most people miss on first look.

Source	examples/fluid/main.mojo, 871 lines, one file
Simulation	320 × 240 cells, magnified ×3
Kernels	six, all Mojo, compiled through this fork's AIR backend
Per frame	~35 <i>dependent</i> GPU dispatches
Buffers	twelve device buffers, one host frame
Presented via	CAMetalLayer, BGRA8

These documents

Chapter	What it covers
1. Where the algorithm comes from	Stable Fluids and its ancestors, and why each inherited decision is in the code
2. The physics	Stable Fluids in the terms the code uses, and why each step is there
3. The six kernels	Every kernel, read line by line, with the decisions inside them
4. A frame, end to end	The dispatch sequence, the buffer choreography, and the run loop around it
5. What to understand	The eight things that will surprise you, and the ones that will bite
6. How this demo came to be	The spike, the launch-cost measurement it was built to take, and the two things it got wrong

The shortest possible summary

A fluid is a velocity field. Move the field along itself, and you have advection. Do that naively and it explodes; do it Stam's way — trace *backwards* from each cell and sample where the fluid came from — and it is unconditionally stable at any time step. That stability is the whole reason the method is famous, and it is four lines of Mojo.

Advection leaves the field compressible: fluid piles up in places. The fix is a projection: compute how much each cell is gaining or losing (divergence), solve for a pressure field that would cancel it, subtract that pressure's gradient. The solve is thirty Jacobi sweeps, which is thirty dispatches, which is why this program is a benchmark as well as a toy.

Then dye — three scalar fields, red, green, blue — rides the corrected velocity, and a final kernel magnifies and tone-maps it into the window.



1. Where the algorithm comes from

Almost every design decision in `main.mojo` is inherited. The comments in the source name the inheritance where it matters — "*Stam's method*", "*one Jacobi sweep of the pressure Poisson equation*", "*turning this into Gauss-Seidel*" — and each of those names is a piece of history that explains why the code looks the way it does. This chapter unpacks them.

Before 1999: fluids that could explode

Computational fluid dynamics is older than computer graphics, and the techniques graphics inherited in the 1990s came with a condition attached.

The natural way to advance a fluid one step is **explicit finite differencing**: compute the derivatives from the current state, step everything forward. It is simple and it is fast, and it is stable only if the time step is small enough that nothing travels more than about one grid cell per step. That is the **CFL condition** (Courant–Friedrichs–Lewy, 1928). Violate it and the simulation does not degrade — it detonates, usually within a handful of frames.

For engineering that is an inconvenience: take smaller steps. For graphics it is fatal. An animator who speeds up a fluid wants it to look faster, not to blow up, and a real-time demo cannot afford to subdivide its time step because the user flicked the mouse.

1999: Stable Fluids

Jos Stam's "**Stable Fluids**" (SIGGRAPH 1999) removed the condition. The paper assembled three ideas, two of them borrowed, into a solver that is **unconditionally stable** — correct-looking at *any* time step.

The borrowed pieces are worth naming, because both are in this program:

Semi-Lagrangian advection, from weather forecasting

Rather than pushing each cell's contents forward, trace **backwards** along the velocity field: for each cell, ask where the fluid arriving here was one step ago, go there, and interpolate what was there. This is the method of characteristics, and it had been standard in **numerical weather prediction** since the 1980s, where the alternative — an explicit scheme on a global atmospheric grid — was ruinously expensive.

It is unconditionally stable for a reason that takes one sentence: every new value is an *interpolation between existing values*, so no value can ever exceed the range of what was already there. There is nothing for a growing mode to grow into.

The source states exactly this, in the docstring of the kernel that does it:

Semi-Lagrangian advection: trace backwards, sample where it came from.

Unconditionally stable whatever the time step, which is the whole reason Stam's method is used here rather than a forward difference.

Projection, from Chorin (1968)

A fluid that does not compress must have a divergence-free velocity field, and advection does not preserve that property. The repair is **Chorin's projection method**: advance the velocity ignoring incompressibility, then project the result back onto the space of divergence-free fields.

That projection is legitimate because of a classical theorem — the **Helmholtz–Hodge decomposition** — which says any vector field splits uniquely into a divergence-free part plus the gradient of a scalar. Compute the scalar (it is the pressure), subtract its gradient, and what is left is divergence-free by construction.

Finding the pressure means solving a Poisson equation, and *that* is where this program spends its frame.

The price: numerical dissipation

Semi-Lagrangian advection averages four neighbours every step, and averaging smears. Stable Fluids loses fine detail — vortices decay that should persist. This was recognised immediately, and the follow-up work is well known: Fedkiw, Stam and Jensen's "**Visual Simulation of Smoke**" (SIGGRAPH 2001) added *vorticity confinement* to inject the lost swirl back, along with a higher-order interpolant to smear less in the first place.

Fluid implements neither, and is right not to. What reads as a defect in a smoke paper reads as viscosity in a dye demo — the billowing, softening plume is the look people expect. It is worth knowing that the softness is the method's error term and not a physical choice.

1845, and why the pressure solve looks like that

The Poisson solve is the expensive part, and the method used is the oldest thing in the program.

Jacobi iteration (Carl Gustav Jacob Jacobi, 1845): to solve a system where each unknown is defined in terms of its neighbours, guess, then repeatedly replace every unknown with what its neighbours currently imply. Every update in a sweep reads the *previous* sweep's values. It converges slowly and it converges reliably.

Gauss–Seidel is the obvious improvement: use each new value as soon as you compute it, rather than waiting for the next sweep. It converges roughly twice as fast, and Stam's own code uses it — the famous ~100-line C listing from his 2003 GDC paper "*Real-Time Fluid Dynamics for Games*", which is the ancestor of most fluid demos ever written, runs Gauss–Seidel.

And it is the wrong choice on a GPU, because "as soon as you compute it" presupposes an order, and thousands of threads running at once do not have one.

This is the single most important comment in the program, and it is a direct consequence of that history:

Ping-ponged rather than updated in place: a Jacobi step reads the previous iterate's whole neighbourhood, and writing in place would feed half-new values back in, silently turning this into Gauss-Seidel with a thread-order-dependent answer.

Write into the buffer you are reading and you do not get Jacobi and you do not get Gauss–Seidel. You get Gauss–Seidel with a **nondeterministic** update order — a solver whose answer depends on GPU scheduling. It still converges. It still looks like a fluid. It differs run to run, machine to machine, and nothing reports it.

The defence is two buffers and a swap. Structural, not careful.

2004: the GPU lineage

The step from Stam's CPU listing to *this* program is also a documented one. Mark Harris's "**Fast Fluid Dynamics Simulation on the GPU**" (*GPU Gems*, 2004) recast Stable Fluids for graphics hardware, and every characteristic shape of this implementation is from that recasting:

Stam, CPU (2003)	GPU (2004 onward), and this program
Gauss–Seidel, in place	Jacobi, ping-ponged between two buffers
a loop over cells	one thread per cell , if <code>idx < N</code>
<code>set_bnd</code> reflecting at walls	a clamped read , stated once in <code>_at</code>
a few sweeps, cheap	thirty dispatches , the bulk of the frame

The boundary simplification is a real one and the source is upfront about it:

The boundary rule is stated once, here: clamp, which lets fluid slide along a wall rather than leak through it.

Stam's `set_bnd` reflects velocity at walls to enforce no-flow-through. A clamp approximates that — the fluid slides rather than bounces — in one line, applied uniformly to every neighbour read in the program. For a demo in an open box it is indistinguishable and it removes an entire category of edge-case bug.

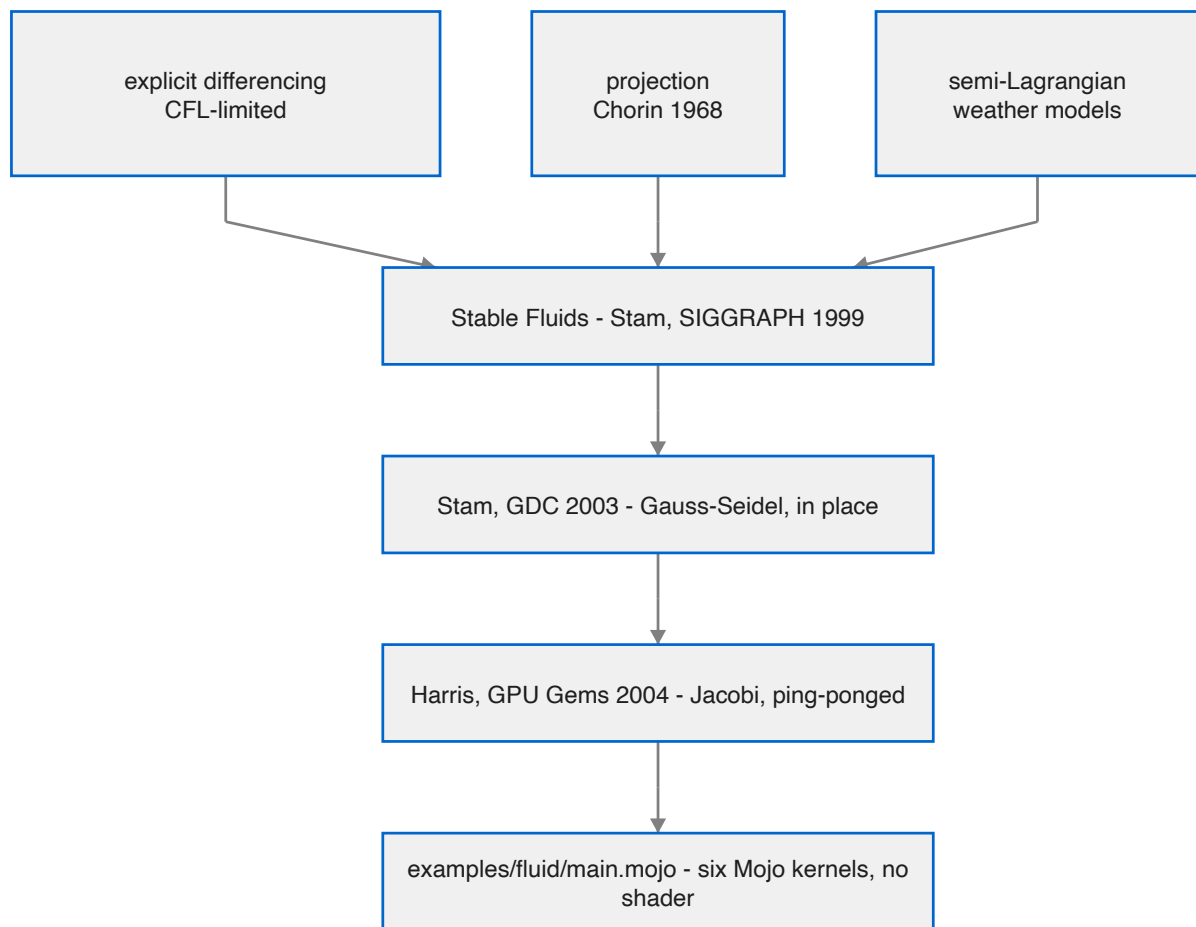
What is not inherited

Two things in the render path are the demo's own, and both are graphics rather than physics:

- **Bilinear magnification** of 320×240 into 960×720, *"the field is smooth, and point-sampling would show the simulation's cells rather than the fluid."*
- **Reinhard tone mapping**, $x / (1 + x)$, *"so a heavy drag saturates gracefully instead of clipping to white."* Dye accumulates without bound; this maps it into $[0, 1)$ smoothly so the image keeps brightening rather than flattening to white.

Neither appears in Stam. Both are doing more for how the demo *looks* than any physics constant in the file.

The lineage, in one picture



2. The physics, in the terms the code uses

You do not need fluid dynamics to read this program, but you do need four ideas. Each maps onto exactly one kernel, which is why the code is short.

What is being simulated

Two things live on a 320×240 grid:

- **A velocity field** — two scalar arrays, `u` and `v`, the horizontal and vertical speed of the fluid in each cell.
- **Dye** — three more scalar arrays, `dr`, `dg`, `db`. The dye does not affect the fluid at all. It is a *tracer*: it goes where the fluid goes, so you can see a field that would otherwise be invisible.

That separation is worth holding on to. The simulation is the velocity field. The colour is passengers.

Idea one: advection, and why it traces backwards

Everything in a fluid is carried along by the fluid, including the fluid itself. That is advection.

The obvious way to compute it is forwards: take what is in this cell, work out where it will be next tick, put it there. This is unstable. Push the time step a little too far and values pile up, feed back, and the simulation explodes — usually within a second, usually spectacularly.

Stam's insight is to run it **backwards**:

For each cell, ask: where was this fluid one time step ago? Go there. Sample what was there. That is what is here now.

```
var px = Float32(idx % W) - dt * u[unsafe_offset=idx]
var py = Float32(idx // W) - dt * v[unsafe_offset=idx]
dst[unsafe_offset=idx] = _bilinear(src, px, py) * fade
```

The departure point almost never lands on a grid cell, so it is **bilinearly interpolated** from the four surrounding cells. And because every new value is an interpolation *between existing values*, it can never exceed the range of what was already there. The method cannot blow up. That is what "unconditionally stable" means, and it is why the source says:

Unconditionally stable whatever the time step, which is the whole reason Stam's method is used here rather than a forward difference.

The price is **numerical diffusion** — averaging four neighbours every step smears the field slightly. In a physics paper that is a defect. In a demo that wants smoke-like billows it looks like viscosity, so nobody minds.

Idea two: incompressibility, and why it needs fixing

Real liquid does not compress. Whatever flows into a region must flow out of it. Advection does not respect this: move a field along itself and you get cells that are gaining more than they lose.

The measure of that failure is **divergence** — for each cell, the net outflow:

```
div = 0.5 * ((u[x+1] - u[x-1]) + (v[y+1] - v[y-1]))
```

Positive means fluid is appearing; negative, disappearing. Both are wrong.

Idea three: projection, and the thirty dispatches

The fix comes from a theorem: **any** velocity field can be split into a divergence-free part plus the gradient of some scalar field. That scalar is *pressure*. Find it, subtract its gradient, and what remains is divergence-free.

Finding it means solving a Poisson equation, $\nabla^2 p = \nabla \cdot u$ — and that is where the dispatches go. The method here is **Jacobi iteration**: repeatedly replace each cell with the average of its four neighbours, minus the divergence:

```
p_next = (p[x-1] + p[x+1] + p[y-1] + p[y+1] - div) * 0.25
```

One sweep barely moves the answer. Thirty gets close enough to look right — `JACOBI_ITERS = 30`, and since the sweeps ping-pong between two buffers, that is **thirty dispatches per frame**, the bulk of the thirty-five.

This is the honest cost of incompressible fluid, and it is the reason Fluid measures launch overhead so well: thirty small, strictly ordered kernels, each of which must complete before the next reads its output.

Then the subtraction, which is the projection proper:

```
u -= 0.5 * (p[x+1] - p[x-1])
v -= 0.5 * (p[y+1] - p[y-1])
```

Idea four: dissipation

Two constants quietly keep the picture alive:

```
comptime DYE_FADE = Float32(0.997)
comptime VEL_FADE = Float32(0.995)
```

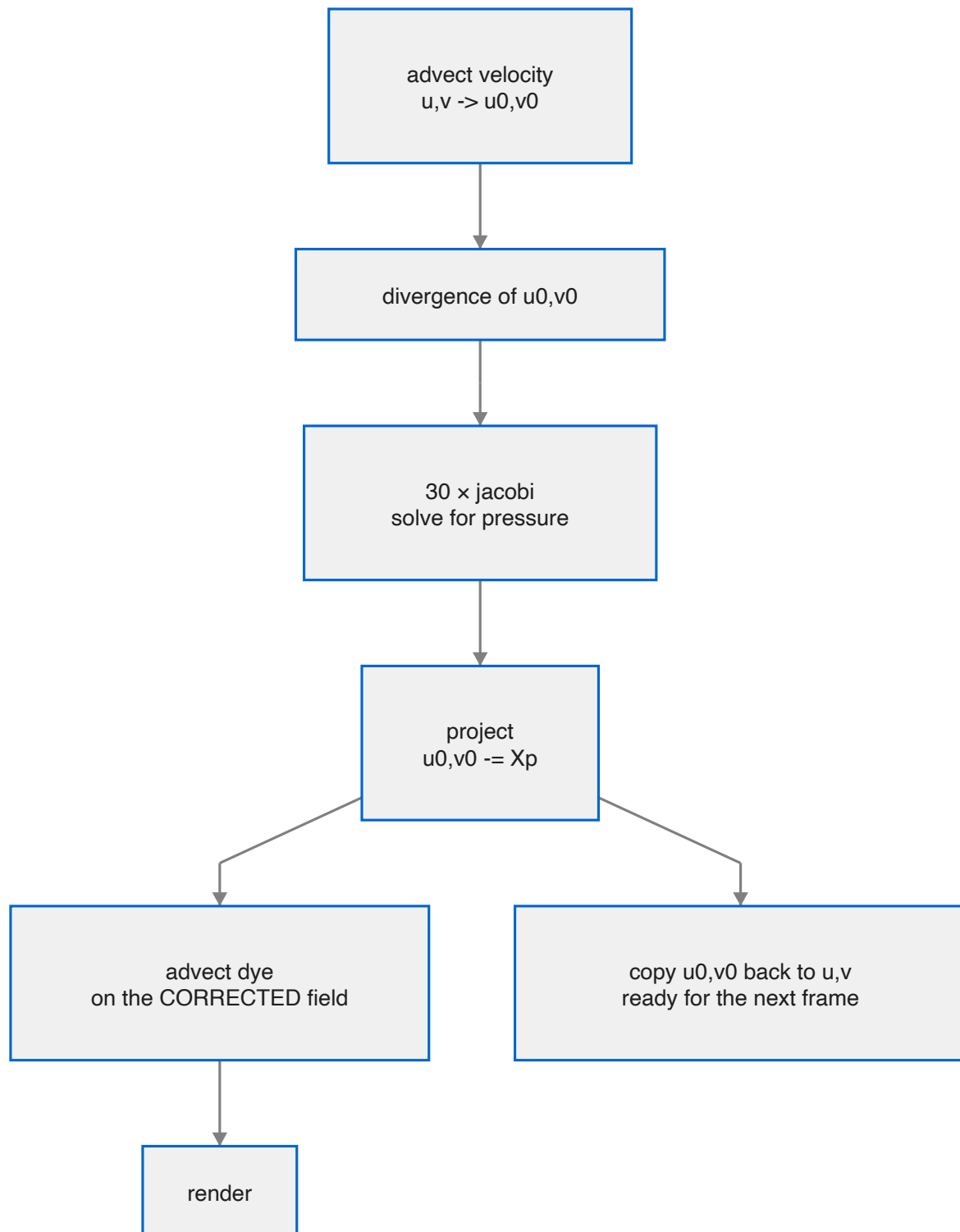
Each is a multiplier applied during advection. Without them, dye accumulates until the screen is uniformly bright and velocity accumulates until the field is noise. With them, a drag makes a plume that

spreads, thins and fades — and the screen returns to black if you stop.

`VEL_FADE` is stronger than `DYE_FADE` on purpose: the motion should die away faster than the colour, so a swirl leaves a visible trail behind it rather than vanishing with its own momentum.

These constants are also what the headless smoke test checks against: dye mass must track `DYE_FADE` to within 0.6%, because advection is supposed to *move* mass, not create or destroy it beyond the fade.

The order, and why it is that order



The one ordering that matters and is easy to get wrong: **dye advects on the corrected field, not the raw one**. If the dye moved on u/v before projection, it would be carried by a compressible field — and

dye would visibly bunch up and thin out in ways the velocity never does. The code advects dye after `project`, using `u0/v0`, for exactly this reason.

3. The six kernels

Every kernel is an ordinary Mojo `def`, compiled through this fork's AIR backend into a `metallib`. There is no shading language anywhere — the same source could run on the CPU, and two of the helpers below deliberately do.

They share one shape:

```
var idx = Int(global_idx.x)
if idx < N:
    ...
```

One thread per cell, flat indexing, a bounds guard because the grid is rounded up to a multiple of `BLOCK = 256`. `N` is 76,800 cells; `GRID` is 300 blocks.

The two helpers everything rests on

`_at` — a clamped read

Neighbour reads at the edge of the grid would run off the array. Every neighbour access goes through a clamp, so the boundary behaves like a wall that the fluid slides along rather than a cliff it falls off.

`_bilinear` — the interpolator

```
var top = a + (b - a) * fx
var bot = c + (d - c) * fx
return top + (bot - top) * fy
```

Four samples, two horizontal lerps, one vertical. This is called by **advection** (to sample a departure point that lands between cells) and by **render** (to magnify 320×240 into 960×720). It is the most-executed code in the program by a wide margin.

`_expf` — and why it is written out by hand

```
@always_inline
def _expf(x: Float32) -> Float32:
    """exp, written out so host and device share one definition.

    Only ever called with x in [-9, 0] (the splat falloff), so the range
    reduction below does not need to be general.
    """
```

A polynomial approximation with a power-of-two scale factor. Two things make it interesting:

1. **Host and device share one definition.** The same function is correct in a kernel and in ordinary CPU code, so the splat looks identical wherever it runs — which is what lets the headless smoke

test check the real thing.

2. **The domain is documented and narrow.** It is only ever called with $x \in [-9, 0]$, so the range reduction does not need to be general. That comment is doing real work: it tells the next person exactly how far they can push this before it stops being true.

1. `advect_kernel` — the heart

```
def advect_kernel(dst: F32, src: F32, u: F32, v: F32, dt: Float32, fade: Float32):
    var px = Float32(idx % W) - dt * u[unsafe_offset=idx]
    var py = Float32(idx // W) - dt * v[unsafe_offset=idx]
    dst[unsafe_offset=idx] = _bilinear(src, px, py) * fade
```

Four lines, and the whole reason the simulation is stable. Note the minus signs: this traces **backwards** in time.

It is called **five times per frame** with different arguments — twice for velocity (`u`, `v`) and three times for dye (`dr`, `dg`, `db`) — which is why `fade` is a parameter rather than a constant. Velocity gets `VEL_FADE`, dye gets `DYE_FADE`.

2. `divergence_kernel` — measuring the error

```
div = 0.5 * ((_at(u, x+1, y) - _at(u, x-1, y))
             + (_at(v, x, y+1) - _at(v, x, y-1)))
```

A central difference in each axis. Six lines, one dispatch, and it produces the right-hand side the pressure solve consumes.

3. `jacobi_kernel` — thirty of the thirty-five

```
def jacobi_kernel(p_next: F32, p: F32, div: F32):
    """One Jacobi sweep of the pressure Poisson equation.

    Ping-ponged rather than updated in place: a Jacobi step reads the previous
    iterate's whole neighbourhood, and writing in place would feed half-new
    values back in, silently turning this into Gauss-Seidel with a
    thread-order-dependent answer.
    """
```

That docstring is the single most important comment in the program. [Chapter 1](#) explains where the hazard comes from — Stam's own code uses Gauss-Seidel, which is faster and inherently sequential, and a GPU has no sequence. Here it is enough to say that writing into the buffer you are reading produces a solver whose answer depends on thread scheduling: it still converges, still looks like a fluid, and differs every run.

The defence is structural, not careful: separate `p` and `p_next` buffers, and a loop that swaps them.

```
for _it in range(JACOBI_ITERS // 2):
    ctx.enqueue_function(jacobi, pr0, pr, div, ...) # pr -> pr0
    ctx.enqueue_function(jacobi, pr, pr0, div, ...) # pr0 -> pr
```

Fifteen iterations of two sweeps — thirty dispatches, and the answer ends up back in `pr` where the projection expects it. The `// 2` is why `JACOBI_ITERS` must be even.

4. project_kernel — subtracting the gradient

```
u[unsafe_offset=idx] -= Float32(0.5) * (_at(p, x+1, y) - _at(p, x-1, y))
v[unsafe_offset=idx] -= Float32(0.5) * (_at(p, x, y+1) - _at(p, x, y-1))
```

Two lines. Everything before it existed to make these two lines meaningful.

Note that this one mutates **in place** — and that is safe, unlike the Jacobi case, because each thread writes only its own cell and reads only `p`, which nothing is writing.

5. splat_kernel — how anything gets in

```
if d2 < r2 * Float32(9.0):
    field[unsafe_offset=idx] += amount * _expf(-d2 / r2)
```

A Gaussian blob added to a field. It is called for the mouse, for the opening puff, and for the `[r]` rain command — and crucially, it is called on **both** kinds of field: dye channels get colour, velocity channels get motion. One kernel, six uses.

The `d2 < r2 * 9.0` test is an early-out at three radii, where the Gaussian is already negligible. Most threads take it, and the exp is never evaluated.

6. render_kernel — the only one that is not physics

```
var sx = Float32(idx % WIN_W) / Float32(SCALE)
var sy = Float32(idx // WIN_W) / Float32(SCALE)
var r = _bilinear(dr, sx, sy)
...
r = r / (Float32(1) + r)
```

Two decisions, both explained in the source:

Bilinear, not nearest. *"the field is smooth, and point-sampling a 320×240 grid into 960×720 would show the simulation's cells rather than the fluid."* The magnification is where the illusion is won or lost.

Tone-mapped with $x/(1+x)$. *"so a heavy drag saturates gracefully instead of clipping to white."* This is Reinhard tone mapping: it maps $[0, \infty)$ into $[0, 1)$ smoothly, so dye can accumulate without ever hitting

a hard white ceiling. Drag hard and the plume gets brighter and *keeps* getting brighter, instead of flattening into a white blob.

Note this kernel runs over `PIXELS` (691,200) rather than `N` (76,800) — it is the only one on the window's grid rather than the simulation's, which is why `PIX_GRID` exists alongside `GRID`.

The count

kernel	dispatches per frame
advect	5 — two velocity, three dye
divergence	1
jacobi	30
project	1
render	1
splat	0–24, only when painting
<i>memset + copies</i>	~6
	~35 fixed

4. A frame, end to end

The program is one file and one loop. This chapter walks a single frame from the mouse to the screen.

Before the loop: twelve buffers and six kernels

```
var u    = ctx.enqueue_create_buffer[DType.float32](N)    # velocity x
var v    = ctx.enqueue_create_buffer[DType.float32](N)    # velocity y
var u0   = ...                                           # velocity scratch
var v0   = ...                                           # velocity scratch
var dr, dg, db = ...                                     # dye, three channels
var s0   = ...                                           # shared dye scratch
var div  = ...                                           # divergence
var pr, pr0 = ...                                       # pressure ping-pong
var frame = ctx.enqueue_create_buffer[DType.uint32](PIXELS)
```

Twelve device buffers, allocated once. Nothing is allocated per frame — a simulation that allocated in its loop would spend more time in the allocator than in the fluid.

Six kernels are compiled once, up front, and reused:

```
var advect  = ctx.compile_function[advect_kernel]()
var diverge = ctx.compile_function[divergence_kernel]()
var jacobi  = ctx.compile_function[jacobi_kernel]()
var project = ctx.compile_function[project_kernel]()
var splat   = ctx.compile_function[splat_kernel]()
var shade   = ctx.compile_function[render_kernel]()
```

Step 1: events

The loop drains the event queue with `nextEventMatchingMask:` and `distantPast` — poll, do not block.

A **drag** does something worth noticing: it splats *five* fields at once.

```
var vx = (mx - last_x) * Float32(6.0)
var vy = (my - last_y) * Float32(6.0)
ctx.enqueue_function(splat, u,  mx, my, 9, vx, ...)    # motion
ctx.enqueue_function(splat, v,  mx, my, 9, vy, ...)
ctx.enqueue_function(splat, dr, mx, my, 7, c[0]*0.6, ...) # colour
ctx.enqueue_function(splat, dg, mx, my, 7, c[1]*0.6, ...)
ctx.enqueue_function(splat, db, mx, my, 7, c[2]*0.6, ...)
```

Three details:

- **The velocity comes from the mouse's own motion**, scaled $\times 6$. You are not pushing a fixed force; you are handing the fluid your gesture. A slow drag makes a gentle plume, a flick makes a jet.

- **The velocity radius (9) is larger than the dye radius (7)**, so the push extends slightly beyond the colour you can see. Without it, dye near the edge of a splat would visibly lag the fluid it is supposed to be in.
- **The hue advances every splat** (`hue += 0.011`), so a session paints through the spectrum rather than one muddy colour.

And the coordinate flip, which is a classic and is commented as such:

```
# Cocoa's origin is bottom-left; the grid's row 0 is the top.
var my = Float32(Float64(WIN_H) - pt.y) / Float32(SCALE)
```

Step 2: the run-loop spin

```
_ = external_call["CFRunLoopRunInMode", Int32](
    P(unsafe_from_address=mode_ref[]), Float64(0.004), Bool(False)
)
```

Four milliseconds. This is the fix described in [chapter 6](#) — without it, Apple Events are delivered to the process and never dispatched, because a hand-rolled `nextEventMatchingMask: pump` services the event queue but not the Mach port.

Step 3: pending commands

Keys and Apple Events both set **bits in one flag**:

```
# Keys set the same flags the Apple Events do, so there is
# one implementation of each verb rather than two.
if kc == 49: g_cmd()[ ] |= CMD_PAUSE
```

`CMD_SNAP` | `CMD_CLEAR` | `CMD_RAIN` | `CMD_PAUSE` | `CMD_QUIT`. Bits rather than booleans, so two commands arriving in one frame cannot lose each other — and one code path per verb regardless of where it came from. The Apple Event handler does nothing but set a bit, because it runs on AppKit's delivery and a GPU dispatch does not belong there.

Step 4: the fluid step, in order

```
ctx.enqueue_function(advect, u0, u, u, v, DT, VEL_FADE, ...) # 1
ctx.enqueue_function(advect, v0, v, u, v, DT, VEL_FADE, ...) # 2
ctx.enqueue_function(diverge, div, u0, v0, ...) # 3
ctx.enqueue_memset(pr, Float32(0))
for _it in range(JACOBI_ITERS // 2):
    ctx.enqueue_function(jacobi, pr0, pr, div, ...) # 4..33
    ctx.enqueue_function(jacobi, pr, pr0, div, ...)
ctx.enqueue_function(project, u0, v0, pr, ...) # 34
```

Note `advect` reads `u, v` and writes `u0, v0` — it cannot advect in place, because a thread reading a neighbour that has already been overwritten would read the future.

Then dye, on the **corrected** field:

```
ctx.enqueue_function(advect, s0, dr, u0, v0, DT, DYE_FADE, ...)
ctx.enqueue_copy(dr, s0)
ctx.enqueue_function(advect, s0, dg, u0, v0, DT, DYE_FADE, ...)
ctx.enqueue_copy(dg, s0)
ctx.enqueue_function(advect, s0, db, u0, v0, DT, DYE_FADE, ...)
ctx.enqueue_copy(db, s0)
```

Three channels through **one shared scratch buffer**, copied back each time. The source is explicit about the trade: *"s0 is the shared scratch, copied back each time, which keeps the buffer count down."* Three extra copies against three extra buffers — a deliberate choice, not an oversight.

Finally the velocity is swapped back for the next frame:

```
ctx.enqueue_copy(u, u0)
ctx.enqueue_copy(v, v0)
```

Step 5: shade and present

```
ctx.enqueue_function(shade, frame, dr, dg, db, grid_dim=(PIX_GRID), ...)
ctx.synchronize()
with frame.map_to_host() as pix:
    for k in range(PIXELS):
        bgra[unsafe_offset+k] = src[unsafe_offset+k]
```

`ctx.synchronize()` is the **one** blocking point in the frame. Everything before it was enqueued; this is where the CPU waits for the GPU to finish the whole batch. That single sync point is precisely why async launch and command- buffer batching matter so much here — thirty-five dispatches queue up and are paid for once.

Then the texture upload and present:

```
_ = send[... , "replaceRegion:mipmapLevel:withBytes:bytesPerRow:"](
    tex, region, Int(0), bgra.unsafe_bitcast[NoneType](), Int(WIN_W * 4))
var cb = send[ObjCObject, "commandBuffer"](queue)
_ = send[ObjCObject, "presentDrawable:"](cb, drawable.ptr())
_ = send[ObjCObject, "commit"](cb)
```

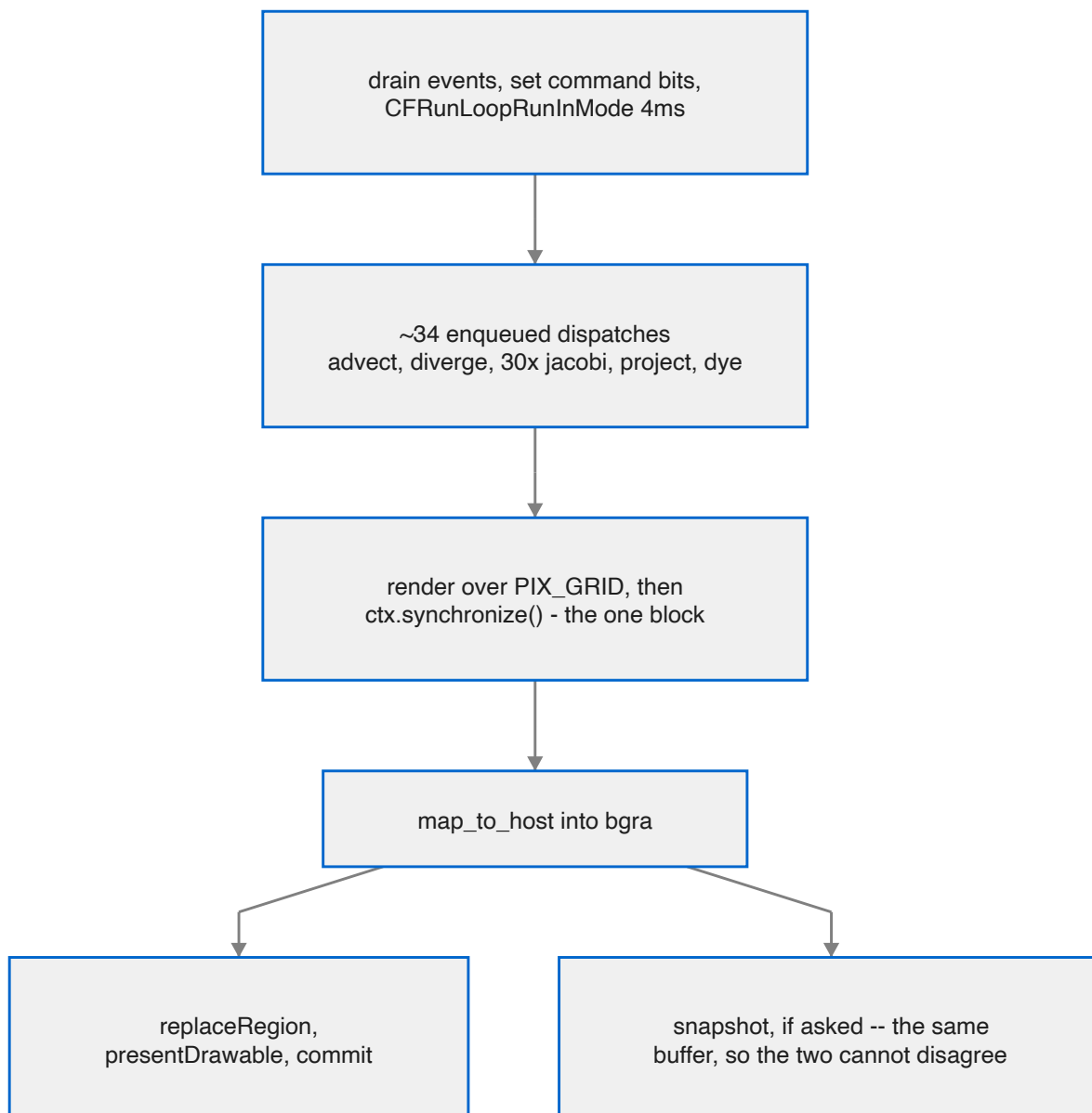
`send` rather than `msg_send` for the Metal protocol objects: the concrete classes behind `MTLTexture` and `MTLCommandQueue` are private, so there is no public class name for the SDK database to check against.

Step 6: the snapshot, if asked

```
# Save here, not later: `bgra` is exactly what is about to be  
# presented, so the file and the window cannot disagree.
```

[s] writes a real PNG — deflate through libz, real CRC — from the same host buffer the layer is about to receive. `FLUID_AUTOSHOT=<frame>` does it headlessly, which is how the demo's own picture was captured on a machine without screen-recording permission.

The whole frame



5. What to understand

Eight things about this demo are not obvious from watching it, and most of them are things that would bite you if you changed the code.

1. It is a benchmark wearing a demo's clothes

The thirty Jacobi dispatches are not a lazily-written solver. They are the *point*. A step of thirty-five **dependent** dispatches is the cheapest honest way to measure per-dispatch launch overhead, and that is why the program was written. If you optimise the pressure solve down to five dispatches, you will have made a nicer fluid and destroyed the instrument.

The switches are the tell:

```
./fluid # async, batched – the default
APPLEGPU_SYNC_LAUNCH=1 ./fluid # the old synchronous bring-up mode
APPLEGPU_BATCH_DISPATCHES=0 ./fluid # async, batching off
```

Those exist so the optimisation this demo motivated can still be turned off and re-measured. Run all three and watch the printed ms/step change by 2.6x.

2. Advection runs backwards, and that is why it never explodes

The single most counter-intuitive line in the program is a minus sign.

```
var px = Float32(idx % W) - dt * u[unsafe_offset=idx]
```

Forwards advection — push each cell's contents to where they are going — is conditionally stable: exceed a time-step limit and it diverges. Backwards advection — ask each cell where its contents *came from*, and interpolate — can only ever produce values between values that already existed. It is **unconditionally stable at any time step**.

`DT = 0.125` is therefore a choice about how fast the fluid looks, not a stability limit you are tiptoeing around. You can raise it and the simulation will get soupy and smeared long before it becomes unstable, because it cannot become unstable.

3. The Jacobi ping-pong is a correctness requirement, not a style

If you take one thing from this walkthrough, take this. From the source:

Ping-ponged rather than updated in place: a Jacobi step reads the previous iterate's whole neighbourhood, and writing in place would feed half-new values back in, silently turning this into

Gauss-Seidel with a thread-order-dependent answer.

Write `pr` while reading `pr` and the program will not crash, will not warn, and will still look like a fluid. It will simply produce a different answer on every run, on every machine, at every occupancy — because which neighbours a thread sees as "already updated" is decided by the GPU scheduler.

This is the shape of GPU bug that costs weeks: no symptom, no reproduction, no stack trace. The defence is structural — two buffers, swapped — rather than careful, and `JACOBI_ITERS` must stay **even** because the loop runs `JACOBI_ITERS // 2` pairs and the answer has to land back in `pr`.

4. Dye rides the *corrected* field

```
ctx.enqueue_function(project, u0, v0, pr, ...)      # first
ctx.enqueue_function(advect, s0, dr, u0, v0, ...)   # then dye, on u0/v0
```

Advect the dye before projection and it is carried by a field that is still compressible — dye will visibly bunch and thin in ways the motion does not explain, and it will look *slightly* wrong in a way nobody can name. Order matters here in a way it does not elsewhere in the frame.

5. `ctx.synchronize()` is the only place the CPU waits

Everything in a frame is `enqueue_`: create, memset, function, copy. One `synchronize()` sits between the render kernel and `map_to_host`, and that is where the entire batch is paid for.

That single sync point is why async launch mattered so much. Adding a second one — say, a debug read-back in the middle of the solve — would not "cost one read"; it would split the batch and reintroduce the round trips the whole design exists to avoid. If you are ever tempted to inspect a buffer mid-frame, use `fluid_smoke.mojo` instead.

6. Magnification is where the illusion lives

The simulation is 320×240. The window is 960×720. Nine screen pixels per simulation cell, and the difference between a fluid and a mosaic is entirely in how those nine are filled:

- **Bilinear, not nearest**, because *"point-sampling a 320×240 grid into 960×720 would show the simulation's cells rather than the fluid."*
- **Reinhard tone mapping**, `r / (1 + r)`, because *"a heavy drag saturates gracefully instead of clipping to white."* Dye can accumulate without limit and the image keeps getting brighter rather than flattening into a white blob.

Both are one line each in `render_kernel`, and both are doing more for the demo's appearance than any physics constant.

7. Nothing here is a shader

There is no Metal Shading Language in this program. The six kernels are ordinary Mojo `def`s compiled through this fork's AIR backend into a `metallib`. Two consequences you can actually use:

- `_expf` is written out by hand so **host and device share one definition** — the same function is correct in a kernel and in CPU code. That is what lets `fluid_smoke.mojo` test the real splat rather than an imitation of it.
- The kernel source and the application source are the same language, the same file, and the same build. There is no shader-compilation step to get wrong.

The Cocoa side is the same story: the window, the `CAMetalLayer`, the Apple Event handler are all Mojo. `FluidAEHandler` is a real Objective-C class declared with `class`, and its `handleEvent:withReplyEvent:` encoding is *derived* from the two object arguments rather than looked up, because that selector is not one the SDK declares.

8. Run the headless twin first

```
./bazel-bin/spikes/fluid/fluid_smoke
```

Same kernels, no window. It reports dye mass against `DYE_FADE`, post-projection divergence, whether velocity is non-zero, and whether a rendered frame is anything but black — and it writes a PPM you can look at.

The source is explicit that these are *observations to read*, *not thresholds it enforces*: 0.6% mass drift and ± 0.05 divergence on a good run. The instruction matters more than the numbers:

Run it first if this ever looks wrong: it separates "the physics broke" from "the window broke".

Those are two completely different investigations, and five seconds of smoke test tells you which one you are in.

A short list of things that will bite

If you change...	...this happens
JACOBI_ITEERS to an odd number	the loop runs one pair fewer and the answer ends up in <code>pr0</code> , which nothing reads
<code>jacobi</code> to write in place	nondeterministic results, no error, no symptom
dye advection to before <code>project</code>	dye bunches and thins for no visible reason
advection to write in place	threads read neighbours from the future
DYE_FADE/VEL_FADE to 1.0	the screen saturates and never recovers
nearest-neighbour sampling in render	a visible 320×240 mosaic
an extra <code>synchronize()</code> per frame	the batch splits; back to the 2.6× slow path
<code>-lz</code> removed from the driver	links under <code>bazel</code> , fails under <code>cocoamojo</code>

Controls

drag	paint dye and push the fluid; the hue advances as you go
space	pause
c	clear
r	rain — random splats
s	save a PNG of exactly the frame about to be presented
<code>fluidctl <pid> snap clear rain pause quit</code>	the same verbs over Apple Events
<code>FLUID_AUTOSHOT=<n></code>	save frame <i>n</i> and quit — headless capture

Keys and Apple Events set the same command bits, so each verb has one implementation regardless of where it came from.

6. How this demo came to be

Fluid was not written because a fluid simulation is nice to look at, though it is. It was written because the fork had no way to measure something it needed to know.

The gap it was built to fill

By August 2026 the fork had two GPU demos, and neither could answer the question that mattered:

- **mandelbrot** is *one* dispatch per frame. Every pixel is independent, the kernel runs once, and the frame is dominated by arithmetic.
- **life** is *none* — the simulation is on the CPU and the GPU only presents.

So neither says anything about **launch cost**: the fixed price of asking the GPU to run a kernel at all, paid once per dispatch regardless of how much work the dispatch does. On a backend as new as this fork's AIR path, that number is exactly what you want to know and exactly what a one-dispatch demo hides.

A Stable Fluids step is about **thirty-five dependent dispatches** — dependent being the operative word, since each must finish before the next can read its output. That makes per-dispatch overhead the dominant term rather than a rounding error, and it makes the program a measuring instrument.

The origin commit says so in as many words:

The point is not that it is nice to look at. mandelbrot is one dispatch per frame and life is none, so neither says anything about launch cost.

What it measured

The first numbers, on an M4:

launch mode	per step	spread
synchronous	10.19 ms	17.31 ms cold — ~70%
APPLEGPU_ASYNC_LAUNCH=1	1.99 ms	±0.2%

5.1x, and the variance collapsed. That works out to 0.234 ms of round trip per dispatch — in the same ballpark as the 0.40 ms measured independently for short kernels, which is the sort of agreement that makes you believe both numbers.

The conclusion was blunt: run synchronously and *the physics alone eats a whole 60 fps frame* before a single pixel is drawn. Not because the arithmetic is heavy — 320×240 is nothing — but because thirty-five round trips are.

Those numbers changed the backend. Async launch became the **default** rather than an opt-in, and command-buffer batching was added to make the slow side faster. On the same workload today:

--	--

	per step
APPLEGPU_SYNC_LAUNCH=1	3.87 ms
default (async + batched)	1.49 ms

2.6× rather than 5.1×, because the floor moved. The conclusion is unchanged: **launch overhead, not arithmetic, is what a 35-dispatch step is made of.**

The switches survive, which is unusual and deliberate — a demo that can turn off the optimisation it motivated is a demo that can prove the optimisation still matters:

```
./fluid                # async, batched – the default
APPLEGPU_SYNC_LAUNCH=1 ./fluid  # the old synchronous bring-up mode
APPLEGPU_BATCH_DISPATCHES=0 ./fluid # async, batching off
```

The headless twin

`spikes/fluid/fluid_smoke.mojo` runs the same kernels with no window and checks them, because "it looked right" is not a claim about a solver. It asserts what a *broken kernel would break*:

- **dye mass** against the dissipation constant — advection conserves mass, so the total must track `DYE_FADE` to within 0.6%
- **post-projection divergence** within ± 0.05 — the projection's entire job
- **velocity non-zero** — a solver that has quietly died is smooth and still
- **a rendered frame that is not entirely black** — the cheapest possible check on the whole pipeline

and it writes a PPM so a human can look. The instruction in the source is worth keeping: *run that first if this ever looks wrong — it separates "the physics broke" from "the window broke"*.

Two things it got wrong, and what fixing them taught

Apple Events, sent and never received

The spike shipped with a control channel — `fluidctl <pid> <verb>` sending a `FLUD` Apple Event for `snap`, `clr`, `rain`, `paus`, `quit` — and a README section headed "**implemented, not yet delivered**". The sending side worked: `fluidctl` built the target descriptor, sent the event, got a non-nil reply and no error. The handler never fired.

The two obvious culprits were tried and changed nothing: calling `-[NSApplication finishLaunching]`, and servicing `CFRunLoopRunInMode`.

The cause was structural, and it is the useful part. Fluid uses mandelbrot's **hand-rolled nextEventMatchingMask: pump** rather than `[NSApp run]`. Apple Event dispatch is something `-[NSApplication run]` *performs*; it is not something the event queue *carries*. `nextEventMatchingMask:` with `distantPast` polls the queue and returns at once — it never services the **Mach port** Apple Events arrive on.

The fix, now in the shipped example, is four lines and a comment that explains itself:

```
# Spin the run loop briefly. `nextEventMatchingMask:` with
# distantPast polls the event queue and returns at once -- it never
# services the Mach port Apple Events are delivered on, so without
# this the handlers registered above are simply never called. Zero
# timeout with returnAfterSourceHandled=true: drain what is ready,
# do not block the frame.
```

A 4 ms `CFRunLoopRunInMode` per frame. The handlers fire.

A missing library, and a build that only worked one way

[s] writes a real PNG — deflated through `libz`, with a real CRC — taken from *the same host buffer that is about to be presented*, so the file and the window cannot disagree. That needs `-lz`, and the spike only ever built under `bazel`, which passed the flag. Built with the driver the IDE's `⌘B` uses, it failed to link.

The fix was to add `-lz` to `cocoamojo` itself, beside `-lobjc` and the frameworks. `libz` ships with macOS at `/usr/lib/libz.dylib`, so it costs nothing but a load command — and a demo that builds under one build system and not the other is a demo that will rot.

The journey to examples/

commit	what changed
1262061c	the spike: Stable Fluids on the GPU, and the launch-cost measurement
365aacf5	fps to stdout — a title bar is invisible to a captured run
3761ce2f	ported to <code>examples/</code> , converted to <code>class</code> , <code>-lz</code> added to the driver
16f0e469	Apple Events and the run-loop spin migrated to the typed-call API
3ad89651	the title as a property, the mode as a bare string

The `class` conversion in `3761ce2f` is the one to notice. The Apple Event target became a *declaration*:

```
class FluidAEHandler:
    def handleEvent_withReplyEvent_(self, event: ObjCObject,
                                    reply: ObjCObject):
```

`handleEvent:withReplyEvent:` is **not** a selector the SDK declares, so its `v@:@@` encoding is *derived* from the two object arguments rather than looked up — the two-argument derivation path, which a separate check confirms both registers and responds to. No `ObjCClassBuilder`, no encoding string, no `cmd` slot.